

Deep Learning using Convolutional Neural Networks

1. Fundamentals

2. Motivation

3. Structure of CNN

4. CNN Regularization and BN Layers

Introduction

CNN is one of the most popular architectures for deep learning nowadays. The background is related to the problem of image classification in computer vision.

CNN provides a powerful image classifier for the CIFAR-10 dataset of images.

CIFAR10 Dataset: Learning Multiple Layers of Features from Tiny Images, Alex Krizhevsky, 2009. Images collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton of the Canadian Institute For Advanced Research, available at:

<https://www.cs.toronto.edu/~kriz/cifar.html>

airplane



automobile



bird



cat



deer



dog



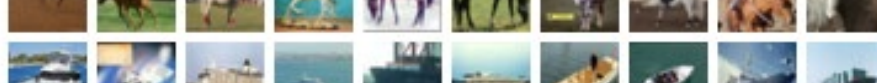
frog



horse



ship



truck



These images are taken in varying lighting conditions and at different angles, and since these are colored images, you will see that there are many variations in the color itself of similar objects.

The CIFAR-10 (Canadian Institute For Advanced Research) dataset consists of 60,000 32 x 32 **colour** images in 10 classes, with 6,000 images per class. There are **50,000 training images and 10,000 test images**. Differences from MNIST are:

- Images are colored as compared to the black and white texture of MNIST.
- Each image is 32 x 32 pixel.
- 50,000 training images and 10,000 testing images.

85% or higher accuracy !

More challenging task

CIFAR-100 is a labeled subset of tiny images dataset. The images were collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton.

The dataset has 100 classes containing 600 images each. There are 500 training images and 100 testing images per class.

Hence, there is a total of 60000 colored images (50000 training and 10000 test) of 32×32 pixels in 100 classes grouped into 20 superclasses. Each image has a fine label (class) and a coarse label (superclass).

<https://paperswithcode.com/sota/image-classification-on-cifar-100>

<https://www.cs.toronto.edu/~kriz/cifar.html>

Image Classification on CIFAR-100

Leaderboard

Dataset

View

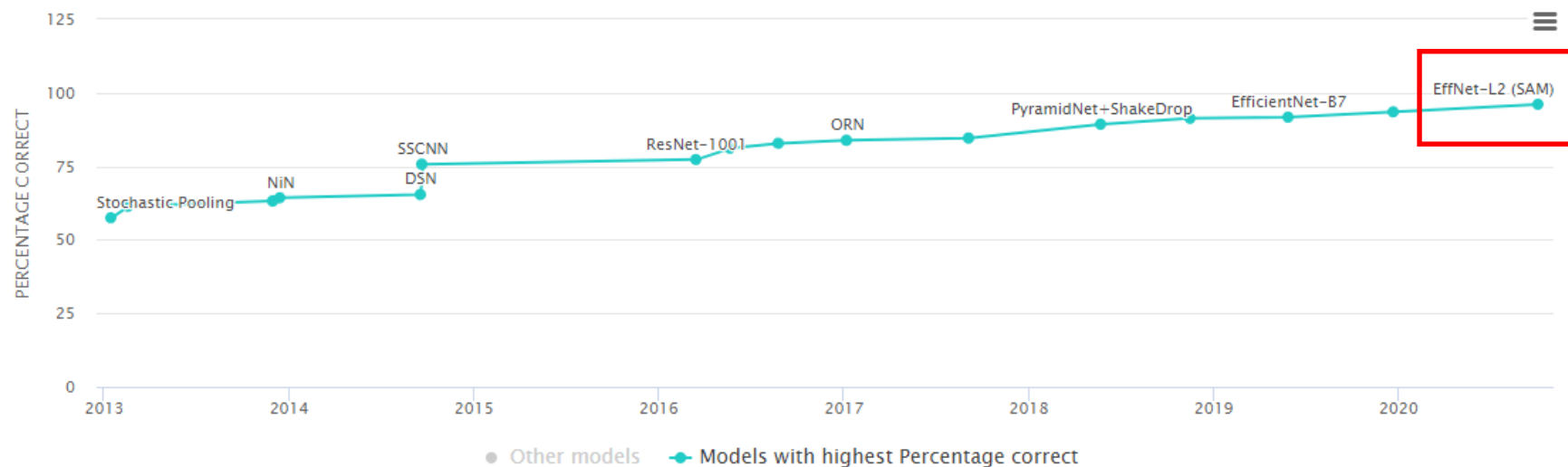
Percentage correct

by

Date

for

All models



Filter:

ResNet

Transformer

EfficientNet

SNN

AlexNet

ResNeXt

Memory-Centric

CLIP Pre-trained

VGG

DCN

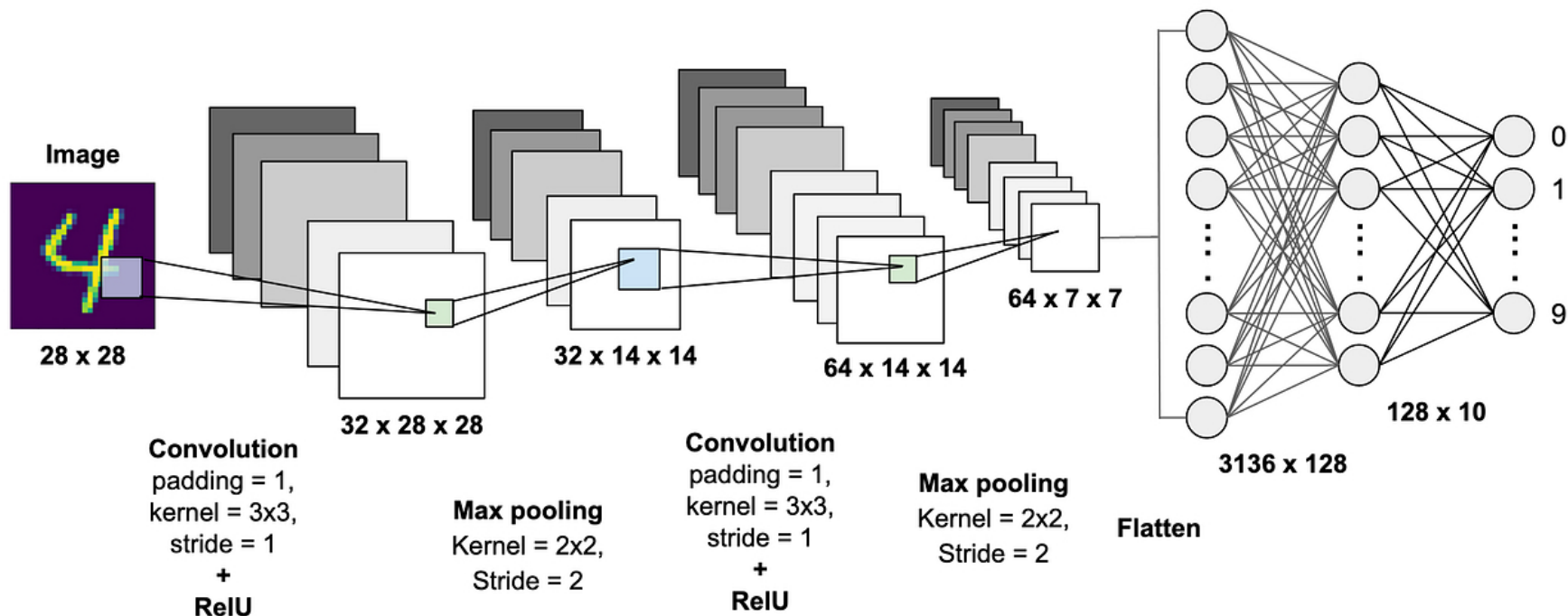
untagged

Edit Leaderboard

Rank	Model	Percentage correct	PARAMS	Accuracy	Extra Training Data	Paper	Code	Result	Year
1	EffNet-L2 (SAM)	96.08			✓	Sharpness-Aware Minimization for Efficiently Improving Generalization	GitHub	Result	2020
2	Swin-L + ML-Decoder	95.1			✓	ML-Decoder: Scalable and Versatile Classification Head	GitHub	Result	2021
3	μ2Net (ViT-L/16)	94.95			✓	An Evolutionary Approach to Dynamic Introduction of Tasks in Large-scale Multitask Learning Systems	GitHub	Result	2022

CNN for deep learning

- The CNNs have several different **filters/kernels** consisting of trainable parameters which can convolve on a given image spatially to **detect features** like edges and shapes.
- Many **filters** are used to learn / capture spatial features from the image based on the **learned weights** obtained through **back propagation**.
- **Many subsequent levels** / layers can be used. The aim is for a given input image to be broken down/“understood” by a **highly abstracted representation** which is then used for prediction in the final fully-connected layer for classification tasks.



Sobel Edge Detection

Input Image:

10	9	9	4	0
0	6	6	2	2
5	9	8	4	3
7	5	5	4	3
8	10	8	5	0

Sobel Kernels

-1	0	1
-2	0	2
-1	0	1

X-Direction

1	2	1
0	0	0
-1	-2	-1

Y-Direction

Before Sobel Filter



After Sobel Filter



How does it differ from a typical ANN ?

- In a regular neural network, we use the entire image to train the network.
- It works well for simple centered image (for example a centered handwritten digit image) but fails to recognize image with more complex variation (for example a running cat in backyard).
- Having more **hidden layers to learn abstract features** would help but it is rather impractical as we need far too many neurons to train and store in memory.
- A CNN/convnet differs from a regular neural network by the use of convolutional layer.

How does CNN work ?

- CNN recognizes a object by first looking for **low level features** such as edges, lines and curves, and then building up more **abstract features** (for example what makes a cat) through series of convolutional layers.
- During the learning process in the convolutional layer, the network **learns individual parts** of an object by shared filters (or feature detectors) on small regions of the image, and add them up to build abstract features.
- **The use of shared filters largely reduce the actual parameter learning.**
- CNN also generalizes better for complex image recognition as **learned filters** can be reused to detect abstract features compositionally through convolutional layers.
- **A typical structure** takes an image, pass it through a series of convolutional layers, nonlinear activation layers, pooling (downsampling) and a fully connected layer to output the classification labels.

Parameters in CNN

1. **Kernel/Filter Size:** A filter is a matrix of weights with which we convolve on the input. The filter on convolution, provides a measure for how close a patch of input resembles a feature.

A feature may be vertical edge or an arch, or any shape. The weights in the filter matrix are learned while training the data.

Smaller filters collect as much local information as possible, bigger filters represent more global, high-level and representative information. Many pixels may be necessary for the network to recognize larger object, and you will use large filters (as 11x11 or 9x9).

Note in general, filters are usually with odd sizes (3x3, 5x5).

Parameters in CNN

2. Padding: Padding is generally used to add columns and rows of zeroes to keep the spatial sizes constant after convolution, doing this might improve performance as it retains the information at the borders. Parameters for the padding function in Keras are :

Same- output size is the same as input size by padding evenly left and right, but if the amount of columns to be added is odd, it will add the extra column to the right.

Valid- Output size shrinks to $(n-f)/s + 1$ where 'n' is input dimensions 'f' is filter size and 's' is stride length.

e.g. $n=6$; $f=3$ (3x3 filter); $s=1$ Then $(6-3)/1 + 1 = 4$

An 6x6 input will result in 4x4 feature map.

Parameters in CNN ...

3. Stride: It is generally the **number of pixels you wish to skip** while traversing the input horizontally and vertically during convolution after each element-wise multiplication of the input weights with those in the filter. It is used to decrease the input image size considerably.

4. Number of Channels: It is the equal to the number of color channels for the input but in later stages is equal to **the number of filters we use for the convolution operation**. The number of channels is the same as the number of filters used. The, it is expected that more features are learnt, but there is a higher chance to over-fit.

5. Pooling-layer parameters: Max-Pooling is generally used among all the pooling options. The objective is to down-sample an input representation (image, hidden-layer output matrix, etc.), reducing its dimensionality by keeping the max value (activated features) in the sub-regions binned.

Hyper-parameters in CNN

Learning rate

Learning rate controls how much to update the weight in the optimization algorithm. We can use fixed learning rate, gradually decreasing learning rate, momentum based methods or adaptive learning rates, depending on our choice of optimizer such as SGD, Adam, Adagrad, AdaDelta or RMSProp.

Number of epochs

Number of epochs is the number of times the entire training set pass through the neural network. We should increase the number of epochs until we see a small gap between the test error and the training error.

Batch size

The samples are used in smaller batches. Typically, networks train faster with mini-batches. A range of 16 to 128 is a typical choice to test with.

Weights Update

When are the kernel parameters updated using back propagation ?

There are three variants for gradient descent algorithm:

- **Gradient Descent**
- **Stochastic Gradient Descent (SGD)**
- **Batch Gradient Descent**

1. **Gradient Descent:** This method passes **the whole data** through the network and finds the error rate for all of them and finds the gradients with respect to all the data samples. Yet, the weights are **updated after passing the whole data-set**. That means for each epoch, only one update occurs. This update is most **accurate** toward descending gradient. (**batch size = all training samples**)
2. **SGD updates the weights after passing each data.** This method is the fastest. (**batch size = 1**). Suppose the dataset has 10,000 items. In one epoch, SGD already has 10,000 parameter updates, whereas the Gradient Descent has only one update.
3. **Batch Gradient Descent is** a trade-off between the above approaches. You specify a batch size and you will **update the weights after passing the data samples in each batch**, means the gradients are calculated after passing each batch. **Suppose you have 1000 data sample and you have specified a batch size = 100. You will have 10 weight update for each epoch.** This method is more accurate than SGD and is faster than the Gradient Descent method.

Hyper-parameters in CNN ...

Activation function

Activation function introduces non-linearity to the model. Usually, rectifier `relu()` works well with CNN. Other alternatives are sigmoid and tanh.

Number of hidden layers and units

It is usually good to add more layers until the test error no longer improves. The trade-off is that it is computationally expensive to train the network. Having a small amount of units may lead to under-fitting while having more units are usually not harmful with appropriate regularization.

Weight initialization

We should initialize the weights with small random numbers to prevent dead neurons, but not too small to avoid zero gradient. Uniform distribution usually works well. The use of command like `(kernel_initializer='normal')` means using **normal distribution**.

Dropout for regularization

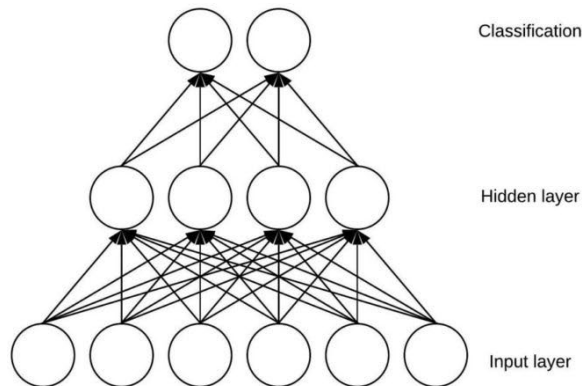
Dropout is a preferable regularization technique **to avoid overfitting** in deep neural networks. The method simply drops out units in neural network according to the desired probability. A default value of 0.3 is a good choice to start with.

Hyper-parameters in CNN ...

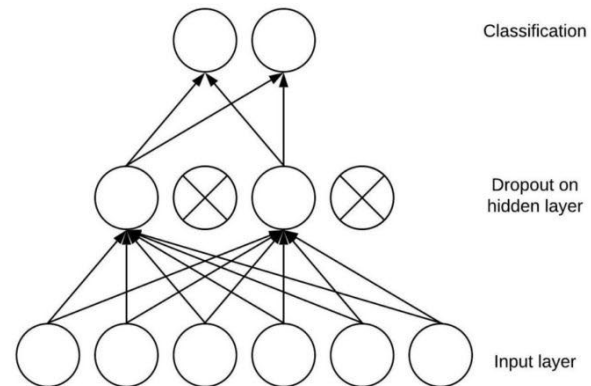
Batch Normalization: Generally in deep neural network architectures, after passing through various adjustments in intermediate layers, **the input may become too big or too small** when it reaches far away layers. This may cause a problem which impacts learning. To solve this problem, we add a batch normalization layer **to standardize (to get zero mean and standard deviation being one) the input** given to the next layer.

Usually, the batch normalization (BN) layer is placed before the sigmoid activation to ensure that the values lie in linear region of sigmoid before the function is applied. However, with `relu()`, the BN is used after.

Dropout: The Dropout layer would nullifies the contribution of some neurons towards the next layer and leaves the rest unmodified. Research has found that dropout layers are useful in training CNNs because they help prevent overfitting on the training data.



Without Dropout



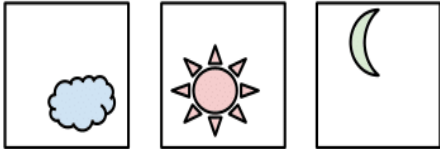
With Dropout

Classification Tasks

Multi-Class Classification Vs Multi-Label Classification Task

Multi-Class

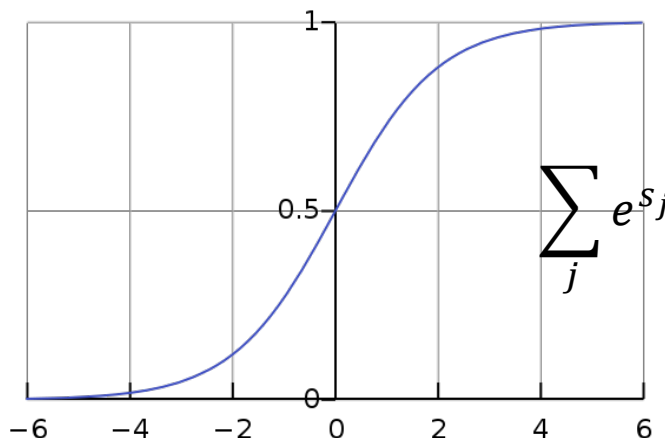
Multi-Label

Multi-Class		Multi-Label	
C = 3	Samples	Samples	
			
	Labels (t) [0 0 1] [1 0 0] [0 1 0]	Labels (t) [1 0 1] [0 1 0] [1 1 1]	

Output Activation Functions

These functions are transformations we apply to vectors coming out from CNNs before the loss computation.

Sigmoid (logistic) function



$$f(s_i) = \frac{1}{1 + e^{-s_i}}$$

Softmax function

The softmax function is constructed out of exponentiation, summation and division operations.

It squashes a vector in the range (0, 1) and all the resulting elements add up to 1. It is applied to the output scores s . As elements represent a class, they can be interpreted as class probabilities.

$$f(s)_i = \frac{e^{s_i}}{\sum_j e^{s_j}}$$

Classification problems

Classification problems are those that involve one or more input variables and the prediction of a class label.

Classification tasks that have just two labels for the output variable are referred to as binary classification problems, whereas those problems with more than two labels are referred to as categorical or multi-class classification problems.

Binary Classification: Task of predicting one of two class labels for a given example.

Multi-Class Classification: Task of predicting one of more than two class labels for a given example.

We can see that the idea of **cross-entropy** may be useful for optimizing a classification model.

Each example has a known class label with a probability of 1.0, and a probability of 0.0 for all other labels. A model can estimate the probability of an example belonging to each class label. **Cross-entropy can then be used to calculate the difference between the two probability distributions.**

Categorical cross entropy loss

Cross entropy loss (some called log loss, negative log-likelihood) measures the performance of the classification model whose output is a probability between 0 and 1.

For a binary classification setting, let t_i be the class label (with binary value of 0 or 1) and s_i be the score from CNN between 0 and 1, for class i . Mathematically, cross entropy is defined by the following equation: ($\log()$ is base e, i.e. natural logarithm)

$$CE = - \sum_i^C t_i \log(s_i)$$

Suppose there are only two classes ($i=1$ and 2), the term **cross entropy loss** is defined as

$$CE = - \sum_{i=1}^{C'=2} t_i \log(s_i) = -t_1 \log(s_1) - (1 - t_1) \log(1 - s_1)$$

(assume $t_2=1-t_1$ and $s_2=1-s_1$)

Cross entropy increases as the predicted probability of a sample diverges from the actual value. For example, when the actual label has a value of 1 and the predicted value is 0.8, the cross entropy loss is calculated as follows:

Expected Class Label = [0 , 1]

Output CNN score = [0.2, 0.8]

Cross-entropy = $-0 * \ln(0.2) - 1 * \ln(0.8) = 0.223$

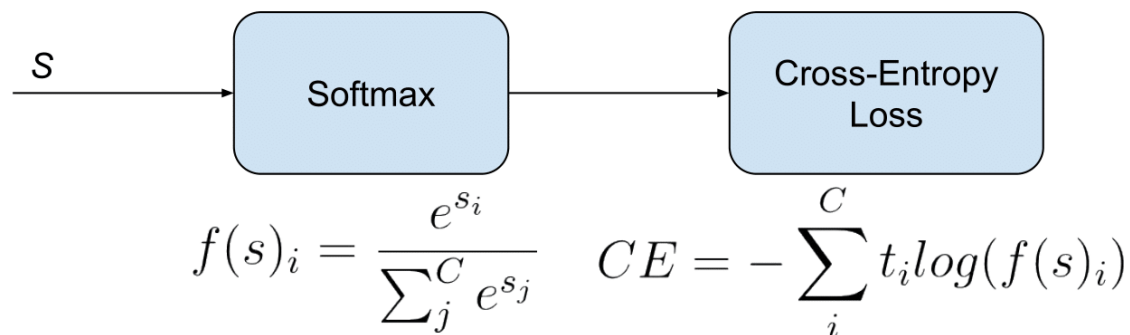
Another example:

Expected Class Label = [0 , 1]

Output CNN score = [0.4, 0.6]

Cross-entropy = $-0 * \ln(0.4) - 1 * \ln(0.6) = 0.511$ (higher value)

When there are many classes (C classes), and softmax function is used to the CNN values before the loss calculation, the loss is called **Categorical cross entropy loss (Softmax Loss)**. If we use this loss, we will train a CNN to output a probability over the C classes for each image. It is used for multi-class classification.



In the specific (and usual) case of multi-class classification the labels are one-hot, so only the positive class S_p keeps its term in the loss. There is only one element of the target vector t which is not zero $t_i = t_p$. So discarding the elements of the summation which are zero due to target labels, we can write:

$$CE = -\log \left(\frac{e^{s_p}}{\sum_j^C e^{s_j}} \right)$$

where S_p is the CNN score for the positive class.

An example:

Expected Class Label = **[0 , 0 , 1 , 0]**

Output CNN score before softmax = **[1.5, 3, 9, 1.5]**

The calculated CNN score after softmax = **[0.1, 0.2, 0.6, 0.1]**

Categorical cross entropy loss = $- [0 * \ln(0.1) + 0 * \ln(0.2) + 1 * \ln(0.6) + 0 * \ln(0.1)]$

$$= -1 * \ln(0.6) = 0.511$$

Cross-Entropy as a Loss Function

Cross-entropy is widely used as a loss function when optimizing classification models.

*Some research has shown that using the cross-entropy error function instead of the sum-of-squares for a classification problem leads to **faster training as well as improved generalization.***

Some principles on building CNN

1. **Always start by using smaller filters.** The aim is to collect as much local information as possible, and then gradually increase the filter width to represent more global, high-level and representative information.
2. Following the principle, **the number of channels should be low (but sufficient) in the beginning such that it detects low-level features.** These are then combined to form many complex shapes (by increasing the number of channels) which help distinguish between classes. **In the later part of the network, the number of filters is increased to increase the depth of the feature space** thus helping in learning more levels of global abstract structures.

Some principles on building CNN...cont'd

By convention the number of channels generally increase or stay the same while we progress through layers in a convolutional neural net architecture

3. General filter sizes used are 3x3, 5x5 and 7x7 for the convolutional layer with a moderate or small-sized images.

For Max-Pooling parameters, we use 2x2 or 3x3 filter sizes with a stride of 2 or 3.

Larger filter sizes and strides may be used to shrink a large image to a moderate size and then go further.

4. Try using **padding = same** when the **border's of the image might be important** or just to help elongate your network architecture as padding keeps the dimensions same even after the convolution operation, and hence one can perform more convolutions without shrinking size.
5. Keep adding layers until you over-fit. Then, we may use dropout, batch normalization, data augmentation etc. to reduce over-fitting.
6. One can use classic networks like LeNet, AlexNet, VGG-16, VGG-19 etc. as an inspiration while building the architectures for your models. Try follow the trend used in those architectures.

For example, trend in the layers Conv-Pool-Conv-Pool or Conv-Conv-Pool-Conv-Conv-Pool or trend in the Number of channels 32–64–128 or 32–32-64–64 or trend in filter sizes, Max-pooling parameters etc.

Demo Example:

CNN architecture: To calculate the
Activation Shape Dataset: MNIST

Example of an CNN architecture:

Task: Classification based on CNN;

3x3convolution (padding = same) + Maxpool,
5x5convolution (padding = valid) +Maxpool;

Flatten to connect to one hidden layer with
120 nodes, then a second hidden layer with 84
nodes, then connect to 10 output nodes.

62430 parameters

CNN architecture: To calculate the Activation Shape

Dataset: MNIST

Example of an CNN architecture:

Layer	Number of Filters	Padding	Activation Shape	Activation Size
Input Image	-	-	(28,28,1)	784
Conv2d(f=3,s=1)	8	Same	(28,28,8)	6,272
MaxPool(f=2,s=2)	-	Valid	(14,14,8)	1568
Conv2d(f=5,s=1)	16	Valid	(10,10,16)	1,600
MaxPool(f=2,s=2)	-	Valid	(5,5,16)	400
Flatten	-	-	(400,1)	400
Flatten	-	-	(120,1)	120
Dense	-	-	(84,1)	84
Softmax	-	-	(10,1)	10

Here, we calculate the Activation shape after every convolution or pooling operation. Dimensions are $((N-f)/s + 1, (N-f)/s + 1, \text{Number of filters})$ wherever the padding value is 'valid'.

Dimensions are $(N, N, \text{Number of filters})$ where the padding used is 'same'.

'N' is input dimensions 'f' is filter size and 's' is stride length.

The Activation Size is computed by multiplying all values in the dimensions of Activation Shape.

For second stage $N=28, f=3$ (3x3 filters), $s=1$, Number of filters=8 and padding is same. Hence, Activation shape is **(28,28,8)**. Activation Size= $28 \times 28 \times 8 = 6272$.

For third stage Max-Pool Layer we get dimension **(14,14,8)**. The last dimension does not change on using a pooling layer. Activation Size = $14 \times 14 \times 8 = 1568$.

For fourth stage $N=14, f=5, s=1$, Number of filters=16 and padding is valid hence we get Activation shape as $(14-5+1), (14-5+1), 16) = \mathbf{(10,10,16)}$. Activation Size = $10 \times 10 \times 16 = 1600$.

For fifth stage Max-Pool Layer we get dimension **(5,5,16)**. Activation Size = $5 \times 5 \times 16 = 400$.

Flatten makes the input into a one-dimensional list for input to the dense layers, therefore Activation shape is **(400,1)**. Activation Size = $400 \times 1 = 400$.

The final stage Dense Activation shape **(84,1)** states 84 hidden units are used in the dense layer. Activation Size = $84 \times 1 = 84$

Next we convert the model architecture to keras code.

The model architecture in keras code:

```
cnn1 = Sequential([
    Conv2D(8, kernel_size=(3, 3),
    activation='relu',padding='same',input_shape=input_shape),
    MaxPooling2D(pool_size=(2, 2),strides=2),
    Conv2D(16, kernel_size=(5, 5), activation='relu'),
    MaxPooling2D(pool_size=(2, 2),strides=2),
    Flatten(),
    Dense(120, activation='relu'),
    Dense(84, activation='relu'),    #64 changed to 84
    Dense(10, activation='softmax')
])
```

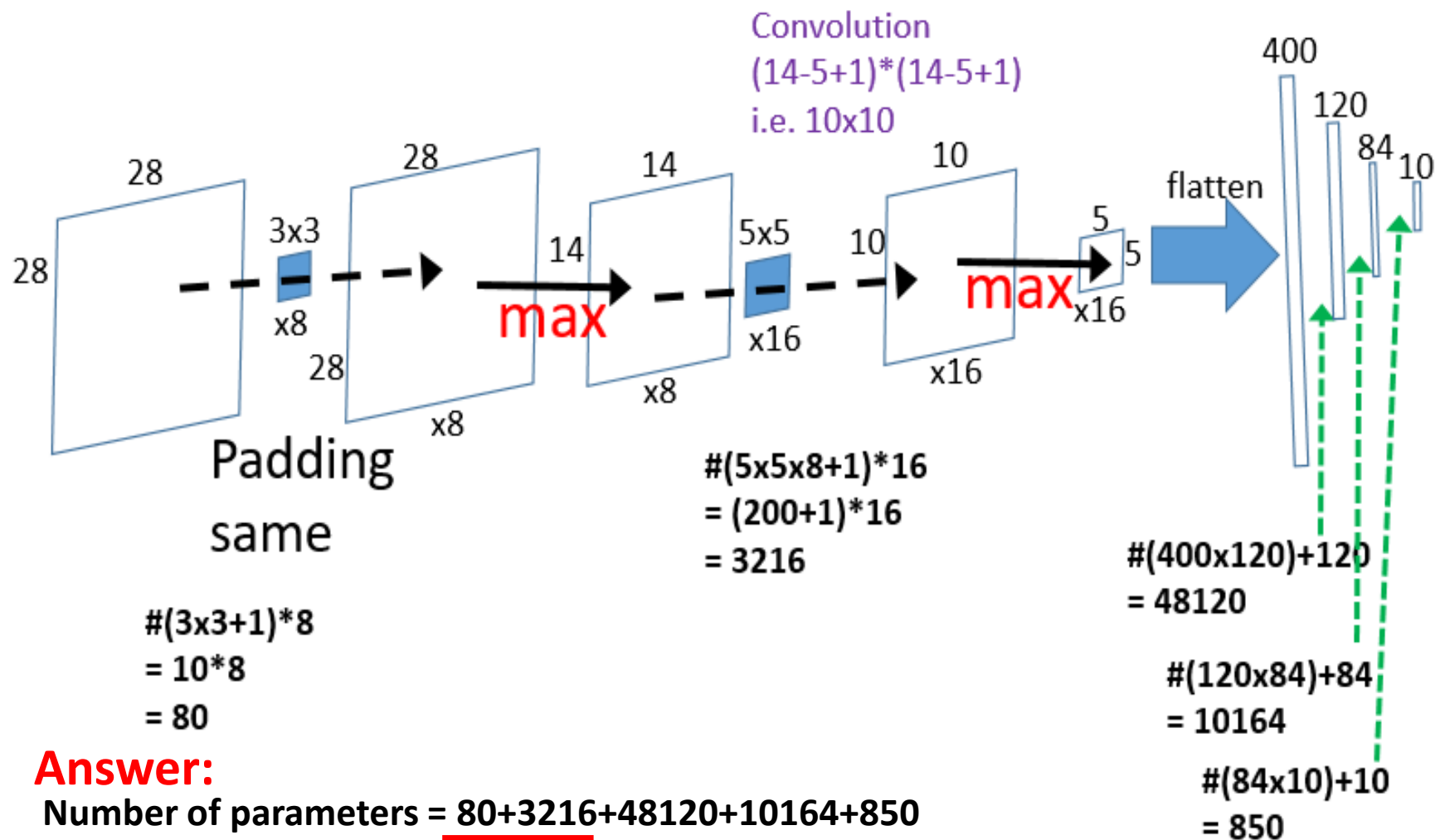
After converting to keras code to check if the conversion is correct and dimensions are the desired ones, one can use <model name>.summary. The **Output Shape** column gives you the output dimensions

```
1 cnn1.summary()
```

Layer (type)	Output Shape	Param #
conv2d_30 (Conv2D)	(None, 28, 28, 8)	80
max_pooling2d_26 (MaxPooling)	(None, 14, 14, 8)	0
conv2d_31 (Conv2D)	(None, 10, 10, 16)	3216
max_pooling2d_27 (MaxPooling)	(None, 5, 5, 16)	0
flatten_10 (Flatten)	(None, 400)	0
dense_20 (Dense)	(None, 120)	48120
dense_21 (Dense)	(None, 84)	10164
dense_22 (Dense)	(None, 10)	850
Total params: 62,430		
Trainable params: 62,430		
Non-trainable params: 0		

Calculations
on the
number of
parameters.

Question: Calculate the number of parameters:



Answer:

Number of parameters = $80 + 3216 + 48120 + 10164 + 850$
= **62430**

Finally an example of a **VGG** like model with the trend Conv-Conv-Pool-Conv-Conv-Pool.

Layer	Number of Filters	Padding	Activation Shape	Activation Size
Input Image	-	-	(28,28,1)	784
Conv2d(f=3,s=1)	16	Same	(28,28,16)	12,544
Conv2d(f=3,s=1)	16	Same	(28,28,16)	12,544
MaxPool(f=2,s=2)	-	Valid	(14,14,16)	3136
Conv2d(f=3,s=1)	32	Valid	(12,12,32)	4,608
Conv2d(f=3,s=1)	32	Valid	(10,10,32)	3,200
MaxPool(f=2,s=2)	-	Valid	(5,5,32)	800
Flatten	-	-	(800,1)	800
Dense	-	-	(512,1)	512
Softmax	-	-	(10,1)	10

Convert to Keras:

```
cnn1 = Sequential([
    Conv2D(16, kernel_size=(3, 3),
    activation='relu',padding='same',input_shape=input_shape),
    Conv2D(16, kernel_size=(3, 3), activation='relu',padding='same'),
    MaxPooling2D(pool_size=(2, 2),strides=2),
    Conv2D(32, kernel_size=(3, 3), activation='relu'),
    Conv2D(32, kernel_size=(3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2),strides=2),
    Flatten(),
    Dense(512, activation='relu'),
    Dense(10, activation='softmax')
])
```

References:

<https://towardsdatascience.com/a-guide-to-an-efficient-way-to-build-neural-network-architectures-part-ii-hyper-parameter-42efca01e5d7>

<https://towardsdatascience.com/a-walkthrough-of-convolutional-neural-network-7f474f91d7bd>

https://gombru.github.io/2018/05/23/cross_entropy_loss/

<https://datascience.stackexchange.com/questions/27421/when-are-weights-updated-in-cnn>